The People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

Yahia Fares University of Medea

Faculty of Sciences

Department of Mathematics and Computer Science

Specialty: Computer Science

AI · Deep Learning · Quran Recitation · Tajweed · Flutter · React

---

Tahkik

AI-Powered Quran Recitation and Tajweed Learning Platform

Mobile Application & Web Administration Dashboard

Supervisor: Mme. *Meriem Ali Khoudja*

By: *Benhadjer Mohamed*
Bouziani Alaa Eddine

Final Year Project (PFE) — L3 Computer Science

Promotion: 2025 / 2026

Medea, Algeria

We dedicate this work to our beloved families,
whose unwavering support and encouragement
have been our greatest source of strength.

To our teachers and mentors,
who guided us through every challenge.

And to every student striving to perfect
the recitation of the Holy Quran.

Acknowledgements

First and foremost, all praise is due to Allah, the Most Gracious, the Most Merciful, for granting us the knowledge, patience, and strength to complete this project.

We would like to express our sincere gratitude to our supervisor, **Mme. Meriem Ali Khoudja**, for her invaluable guidance, constructive feedback, and continuous encouragement throughout the development of this project.

We are deeply grateful to our families for their unconditional love, support, and understanding during the intensive months of work on this project.

We also extend our thanks to all the professors and staff of the Department of Computer Science at Yahia Fares University of Medea for the quality education and resources they have provided throughout our academic journey.

Finally, we would like to thank our friends and colleagues who contributed, whether through testing, feedback, or moral support, to the success of this project.

Abstract

Abstract

Our final study project is a mobile and web platform dedicated to Quran recitation and Tajweed learning powered by Artificial Intelligence. Tahkik aims to bridge the gap between traditional Quranic education and modern technology by providing learners with instant AI-driven feedback on their recitation, while empowering teachers with a comprehensive dashboard to review and manage student progress. The mobile application, built with Flutter, allows users to read the Quran, record their recitation, and receive real-time pronunciation analysis using the Whisper AI model. The web dashboard, developed with React and TypeScript, provides teachers and administrators with tools for recitation review, student management, and platform analytics. The system uses Supabase as its backend-as-a-service for authentication, database, and real-time capabilities.

Keywords: Quran, Tajweed, AI, Speech Recognition, Whisper, Flutter, React, Supabase, Mobile Application, E-Learning.

Résumé

Notre projet de fin d'études est une plateforme mobile et web dédiée à l'apprentissage de la récitation du Coran et du Tajweed, alimentée par l'Intelligence Artificielle. Tahkik vise à combler le fossé entre l'éducation coranique traditionnelle et la technologie moderne en fournissant aux apprenants un retour instantané généré par l'IA sur leur récitation, tout en offrant aux enseignants un tableau de bord complet pour examiner et gérer les progrès des étudiants. L'application mobile, développée avec Flutter, permet aux utilisateurs de lire le Coran, d'enregistrer leur récitation et de recevoir une analyse de prononciation en temps réel. Le tableau de bord web, développé avec React et TypeScript, fournit aux enseignants et administrateurs des outils de gestion.

Mots-clés: Coran, Tajweed, IA, Reconnaissance Vocale, Whisper, Flutter, React, Supabase, Application Mobile, E-Learning.

Contents

List of Figures

List of Tables

General Introduction

The recitation of the Holy Quran holds a central place in Islamic practice and spirituality. For over fourteen centuries, the science of Tajweed — the set of rules governing the correct pronunciation of Quranic verses — has been transmitted orally from qualified scholars to students through direct, face-to-face instruction. This traditional method, known as *Talaqqi* and *Mushafahah*, remains the gold standard for learning proper recitation.

In recent years, the rapid advancement of Artificial Intelligence (AI), particularly in the fields of Automatic Speech Recognition (ASR) and Natural Language Processing (NLP), has opened new possibilities for educational technology. Models such as OpenAI's Whisper have demonstrated remarkable capabilities in transcribing and understanding spoken language across multiple languages, including Arabic. These technological breakthroughs present a unique opportunity to develop intelligent tools that can assist learners in perfecting their Quranic recitation.

Problem Statement

Despite the abundance of digital Quranic resources, most existing applications function merely as digital copies of the Mushaf or provide pre-recorded recitations for passive listening. Very few platforms offer interactive, real-time feedback on a user's own recitation quality. The shortage of qualified Tajweed teachers, especially for non-Arabic speakers and those in remote areas, further limits access to personalized instruction. Furthermore, no existing platform effectively combines AI-driven instant feedback with a structured teacher review system, leaving a significant gap in the Quranic e-learning ecosystem.

The main objectives of this project are:

1. To develop a cross-platform mobile application (using Flutter) that enables users to read the Quran and record their recitation for AI-based evaluation.
2. To integrate the Whisper AI model for real-time pronunciation analysis and Tajweed error detection.
3. To build a web-based administration and teacher dashboard (using React) for managing students, reviewing recitations, and providing manual feedback.
4. To create a comprehensive learning ecosystem with progress tracking, practice

sessions, and subscription management.

To achieve these objectives, we adopted an Agile-inspired development methodology, working in iterative sprints with continuous review and improvement. The system architecture follows a client-server model, with the Flutter mobile app and React web dashboard communicating with a Supabase backend for authentication, database operations, and real-time capabilities, while a dedicated Python server handles AI inference.

This document is organized as follows:

- **Chapter 1 — State of the Art**: Presents the domain of Quranic recitation and Tajweed, reviews existing digital solutions, and identifies the gap that Tahkik addresses.
- **Chapter 2 — Conceptual Study**: Details the functional and non-functional requirements and presents the system design through UML diagrams (Use Case, Sequence, Class, and Architecture diagrams).
- **Chapter 3 — Implementation and Realization**: Describes the development environment, technology stack, database design, and showcases the application's user interfaces.

1. State of the Art

1.1 Introduction

In order to provide an effective and relevant solution, it is essential to study and analyze the existing facts, practices, and technologies related to Quranic recitation learning and digital platforms. This analysis helps identify key elements and existing trends that will inform and guide the development of our proposed solution.

This chapter presents an overview of the current context surrounding Quranic e-learning. It begins with a general definition of Tajweed and its importance, followed by a review of existing digital solutions. It concludes with a comparative analysis that highlights the gap our project aims to fill.

1.2 Domain Overview

1.2.1 Definition and Context

Tajweed

Tajweed literally means "to make well" or "to improve." In the context of Quranic recitation, it refers to the set of linguistic and phonetic rules applied when reading the Quran. These rules govern the correct pronunciation of each letter from its proper articulation point (*Makhraj*) with its associated attributes (*Sifat*). Proper application of Tajweed is considered obligatory (*Fard*) upon every Muslim who recites the Quran, as incorrect pronunciation can alter the meaning of the sacred text.

1.2.2 Traditional Learning Methods

Traditionally, Tajweed is learned through direct interaction with a qualified teacher (*Sheikh*). This process involves two key methods:

- **Talaqqi (Receiving)**: The student listens to the teacher recite, absorbing the correct pronunciation and rhythm.
- **Mushafahah (Lip-to-lip)**: The student recites back to the teacher, who observes the mouth movements and corrects pronunciation in real-time.

While highly effective, this traditional method faces several modern challenges:

- **Accessibility**: A critical shortage of qualified teachers, especially in non-Arabic speaking regions and remote areas.
- **Time Constraints**: Scheduling regular sessions with a teacher is difficult for individuals with busy lifestyles.
- **Scalability**: A single teacher can only attend to a limited number of students effectively at any given time.
- **Cost**: Private tutoring can be expensive and unaffordable for many learners.

1.2.3 Key Concepts

Automatic Speech Recognition (ASR)

Automatic Speech Recognition is the computational task of converting spoken language into written text. Modern ASR systems use deep learning models trained on large datasets of audio-text pairs to achieve high accuracy across multiple languages and dialects.

Whisper Model

Whisper is a general-purpose speech recognition model developed by OpenAI. Trained on 680,000 hours of multilingual data, it demonstrates strong performance in transcription, translation, and language identification. Its support for Arabic makes it a promising candidate for Quranic recitation analysis.

1.3 Existing Solutions

With the advent of smartphones and the internet, numerous applications have been developed to facilitate Quran reading and memorization. We analyze the most relevant existing solutions below.

1.3.1 Quran.com / Ayat

Key Features of Quran.com

- Digital Mushaf with multiple translations
- Professional audio recitations by renowned Qaris
- Verse-by-verse audio playback and bookmarking
- Word-by-word translation and transliteration

Strengths: Excellent for reading, listening, and following along. High accessibility and offline support.
Weaknesses: Completely passive — no interactive feedback. Users can listen to correct pronunciation but receive no evaluation of their own recitation.

1.3.2 Tarteel AI

Key Features of Tarteel AI

- AI-powered recitation tracking (follows along as user recites)
- Mistake detection during memorization (Hifz)
- Gamification elements (streaks, challenges)
- Voice search for Quranic verses

Strengths: Real-time AI tracking, gamification to increase engagement, modern user interface.
Weaknesses: Focuses primarily on memorization (whether the user said the *right word*) rather than on precise phonetic Tajweed rules (how the user *pronounced* the word). No structured teacher involvement or human review system.

1.3.3 Live Tutoring Platforms (Online Academies)

Key Features of Online Tutoring

- One-on-one sessions with qualified teachers via video conferencing
- Personalized curriculum and feedback
- Scheduling flexibility

Strengths: Provides personalized, expert-level feedback from human teachers.
Weaknesses: High cost per session, scheduling dependencies, limited availability of highly qualified teachers, and no AI-assisted pre-screening of recitations.

1.4 Comparative Analysis

Table 1.1: Comparison of existing solutions

Criteria	Quran.com	Tarteel AI	Online Tutoring	Tahkik
Free Access	✓	×	×	✓
AI Recitation Feedback	×	✓	×	✓
Tajweed Rule Detection	×	×	✓	✓
Teacher Dashboard	×	×	×	✓
Progress Tracking	×	✓	✓	✓
Human Review System	×	×	✓	✓
Mobile App	✓	✓	×	✓
Cross-Platform	✓	✓	×	✓

1.5 Problem Statement

The analysis of existing solutions reveals a clear gap in the market: there is no platform that effectively combines the **scalability and instant feedback of AI** with the **nuanced, authoritative guidance of human teachers** in a single, accessible ecosystem.

Identified Limitations

- Existing AI tools focus on word-level tracking, not phonetic Tajweed rule application.
- No platform provides a dedicated teacher dashboard for reviewing AI-flagged recitations.
- There is no unified system combining AI feedback with structured human review.
- Most solutions lack comprehensive progress tracking and structured learning paths.

1.6 Proposed Solution

To address these limitations, we propose **Tahkik** — an AI-powered platform for Quran recitation and Tajweed learning.

Project Objectives

1. Provide instant AI-driven pronunciation feedback using the Whisper model
2. Offer a dedicated teacher dashboard for expert recitation review (human-in-the-loop)
3. Create a structured learning ecosystem with progress tracking and practice sessions
4. Support cross-platform access through a Flutter mobile app and React web dashboard

1.7 Conclusion

In this chapter, we presented the domain of Quranic recitation and Tajweed, reviewed three categories of existing digital solutions, and identified their key limitations. These findings motivate the design of our solution, Tahkik, which will be detailed in the conceptual study presented in Chapter 2.

2. Conceptual Study

2.1 Introduction

This chapter presents the analysis and design phase of the Tahkik project. We use the Unified Modeling Language (UML) to model the system's requirements, behavior, and structure. The chapter begins with requirements identification, proceeds to behavioral modeling through use case and sequence diagrams, and concludes with the static structure via class diagrams and database schema design.

2.2 System Development Process

Our development process is inspired by Agile Methodology, based on flexibility, iterative development and continuous feedback. Our workflow follows these practices:

Phase 1 — Backlog Management

We maintain a flexible list of features and tasks. Every 10 to 15 days, we select a set of prioritized tasks forming our Sprint Backlog, including implementation (mobile and web), UI/UX design, AI integration, and system conception.

Phase 2 — Sprint Cycle (1–2 Weeks)

Each sprint focuses on producing a specific feature. The team plans upcoming tasks, implements them based on each member's role, reviews completed work, and provides feedback for improvement.

Phase 3 — Continuous Review

We conduct continuous informal reviews where each member presents their work, and the rest of the team provides constructive feedback on quality and direction.

Phase 4 — Incremental Production

At the end of each sprint, a part of the platform is complete and integrated, leading to a fully functional product through incremental releases.

2.3 Requirements Analysis

2.3.1 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement	Description
FR-01	Authentication	Users must be able to register, log in (email/password or Google Sign-In), and log out securely.
FR-02	Quran Reading	Learners can browse and read the Quran with Warsh script rendering.
FR-03	Recitation Recording	Learners can record their voice reciting selected verses.
FR-04	AI Feedback	The system processes audio using Whisper AI and returns a confidence score with error detection.
FR-05	Teacher Review	Teachers can listen to submitted recitations, view AI feedback, and provide manual corrections.
FR-06	Progress Tracking	Learners can view their recitation history, scores, and improvement trends.
FR-07	Student Management	Teachers can manage their assigned students and track individual progress.
FR-08	Subscription	The system handles different subscription tiers for premium features.
FR-09	Ijaza Certification	Teachers can issue digital Ijaza certificates to qualifying students.
FR-10	Platform Analytics	Admins can view overall platform statistics (users, recitations, AI health).

2.3.2 Non-Functional Requirements

Table 2.2: Non-functional requirements

ID	Category	Description
NFR-01	Performance	AI feedback should return results within a few seconds of audio submission.
NFR-02	Security	User data and audio must be securely stored using JWT and RLS policies.
NFR-03	Usability	The mobile interface must be intuitive for users of all ages and technical levels.
NFR-04	Scalability	The backend must handle concurrent audio processing requests.
NFR-05	Reliability	The platform should maintain high availability with minimal downtime.

2.4 Use Case Modeling

Use Case Diagram (UCD)

A use case diagram is a type of UML diagram that represents functional requirements. It visually describes interactions between users (actors) and the system, clarifying the operations and actions they can perform.

2.4.1 Actor Identification

Table 2.3: System actors and their roles

Actor	Role Description
Learner	A registered user who practices Quran recitation, receives AI and teacher feedback, and tracks progress.
Teacher	A qualified instructor who reviews student recitations, provides manual feedback, manages students, and issues Ijaza certificates.
Admin	A system supervisor with full control over the platform, managing users, analytics, and AI health.
Tahkik AI Server	The backend AI system (Whisper model) that processes audio and generates automated feedback.
Supabase	The backend-as-a-service handling authentication, database, and real-time subscriptions.

2.4.2 Global Use Case Diagram

The following use case diagram illustrates the global functionalities of the Tahkik platform across all actors.

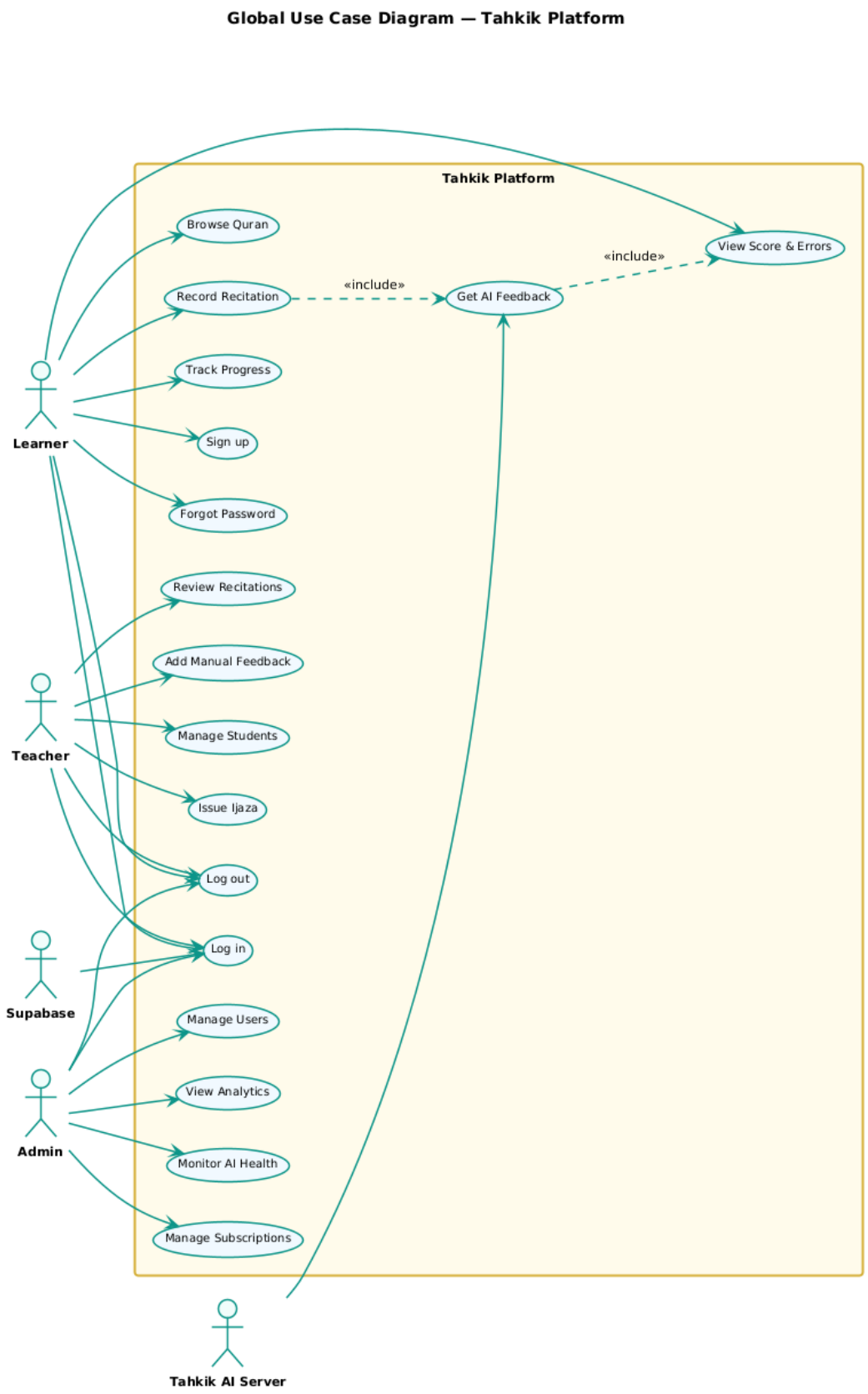


Figure 2.1: Global Use Case Diagram for the Tahkik Platform

Table 2.4: UCD of Authentication description

Action	Actors	Description
Sign up	Visitor	User creates an account by filling name, email, password, and selecting preferences.
Log in	Learner, Teacher	User logs in with email/password or via Google Sign-In.
Forgot password	Visitor	Initiates a password recovery process via email.
Log out	All users	User logs out and session is terminated.

2.4.3 Use Case Diagram: Quran Recitation and AI Feedback

The following use case diagram illustrates the core recitation and AI feedback features.

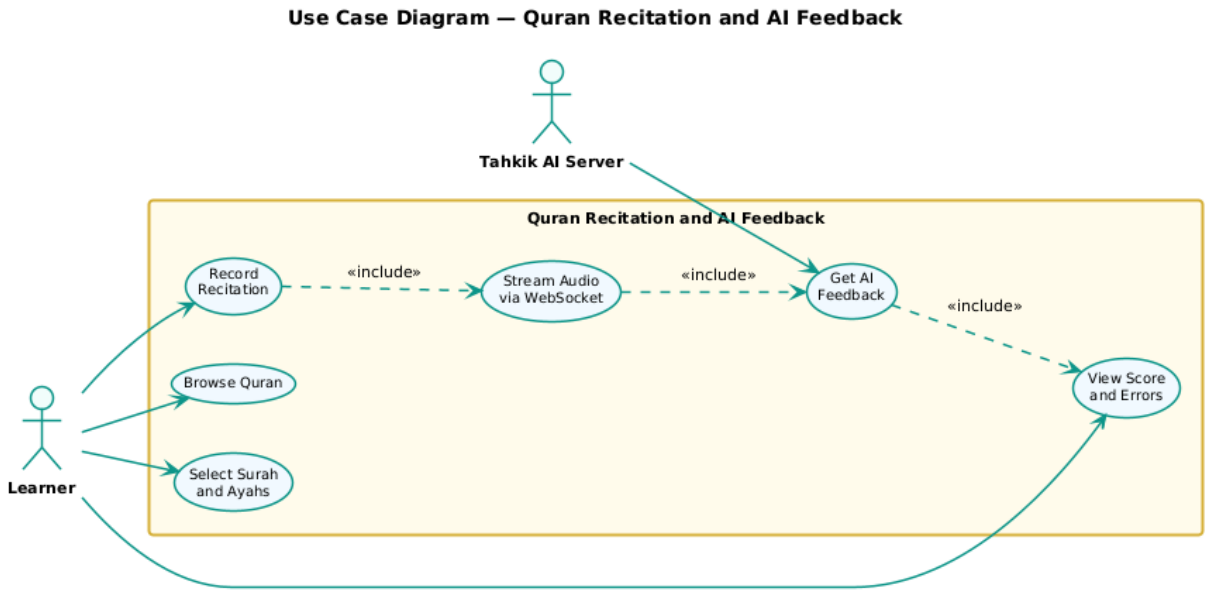


Figure 2.2: Use case diagram for Quran Recitation and AI Feedback

Table 2.5: UCD of Quran Recitation description

Action	Actors	Description
Browse Quran	Learner	Learner navigates the Quran reader with Warsh font rendering.
Record Recitation	Learner	Learner records audio of their voice reciting selected verses.
Submit Audio	Learner	Audio is streamed via WebSocket to the AI Server for processing.
Get AI Feedback	Tahkik AI	Whisper model transcribes audio, detects errors, returns confidence score.
View Score	Learner	Learner views the AI-generated feedback with highlighted errors.

2.4.4 Use Case Diagram: Teacher Review and Student Management

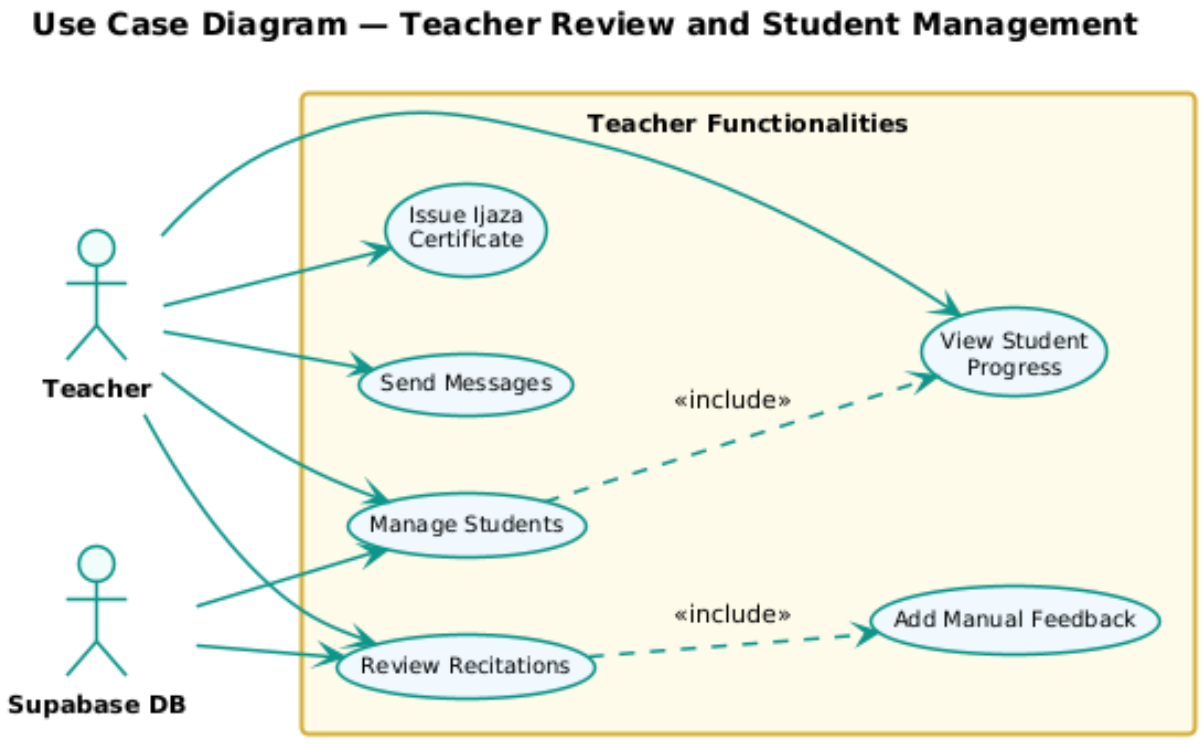


Figure 2.3: Use case diagram for Teacher Review and Student Management

Table 2.6: UCD of Teacher Review description

Action	**Actors**	**Description**
Review Recitations	Teacher	Teacher views a queue of pending student recitations with AI flags.
Add Manual Feedback	Teacher	Teacher listens to audio, reviews AI output, and adds corrections.
Manage Students	Teacher	Teacher views student list, progress, and individual details.
Issue Ijaza	Teacher	Teacher issues a digital Ijaza certificate to qualifying students.
Send Messages	Teacher	Teacher communicates with students through in-app messaging.

2.4.5 Use Case Diagram: Administration

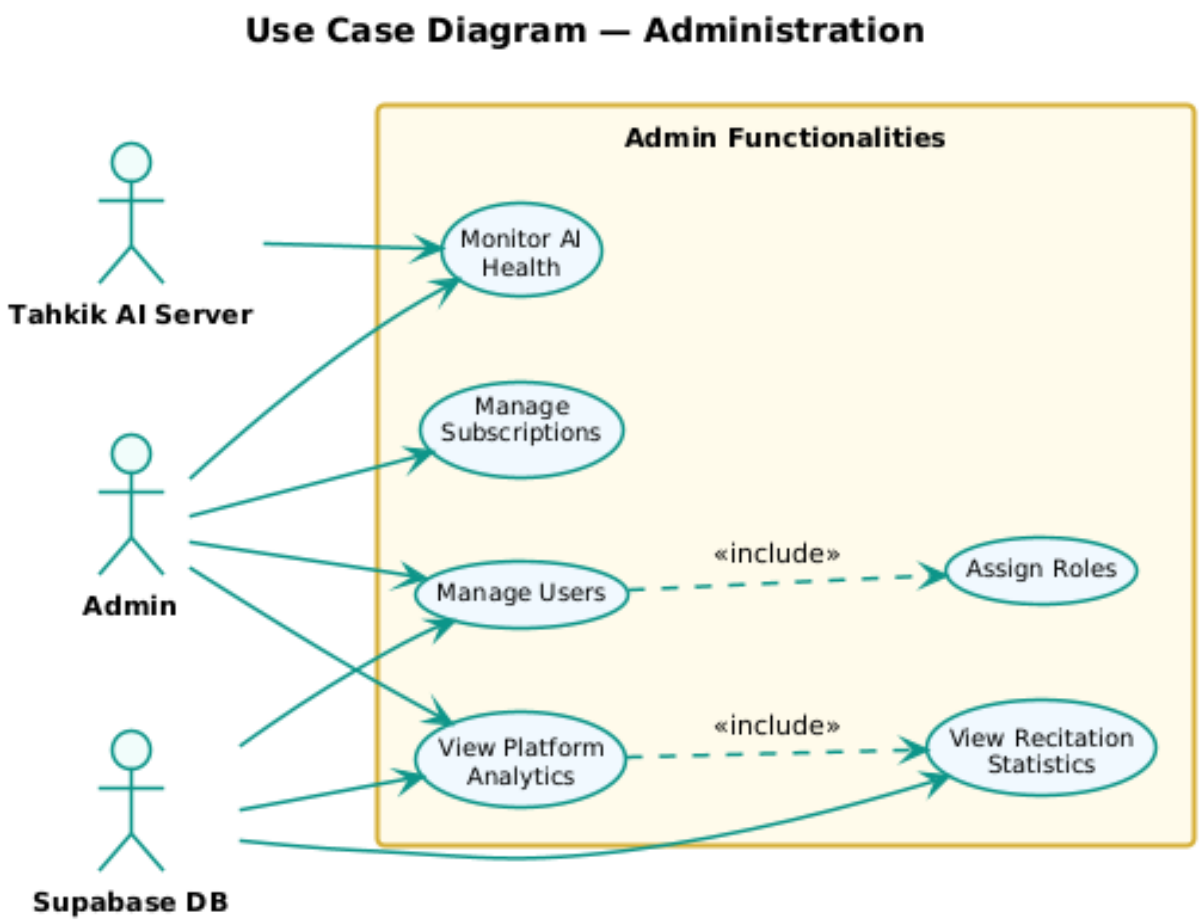


Figure 2.4: Use case diagram for Administration

2.4.6 Use Case Diagram: Progress Tracking and Subscription

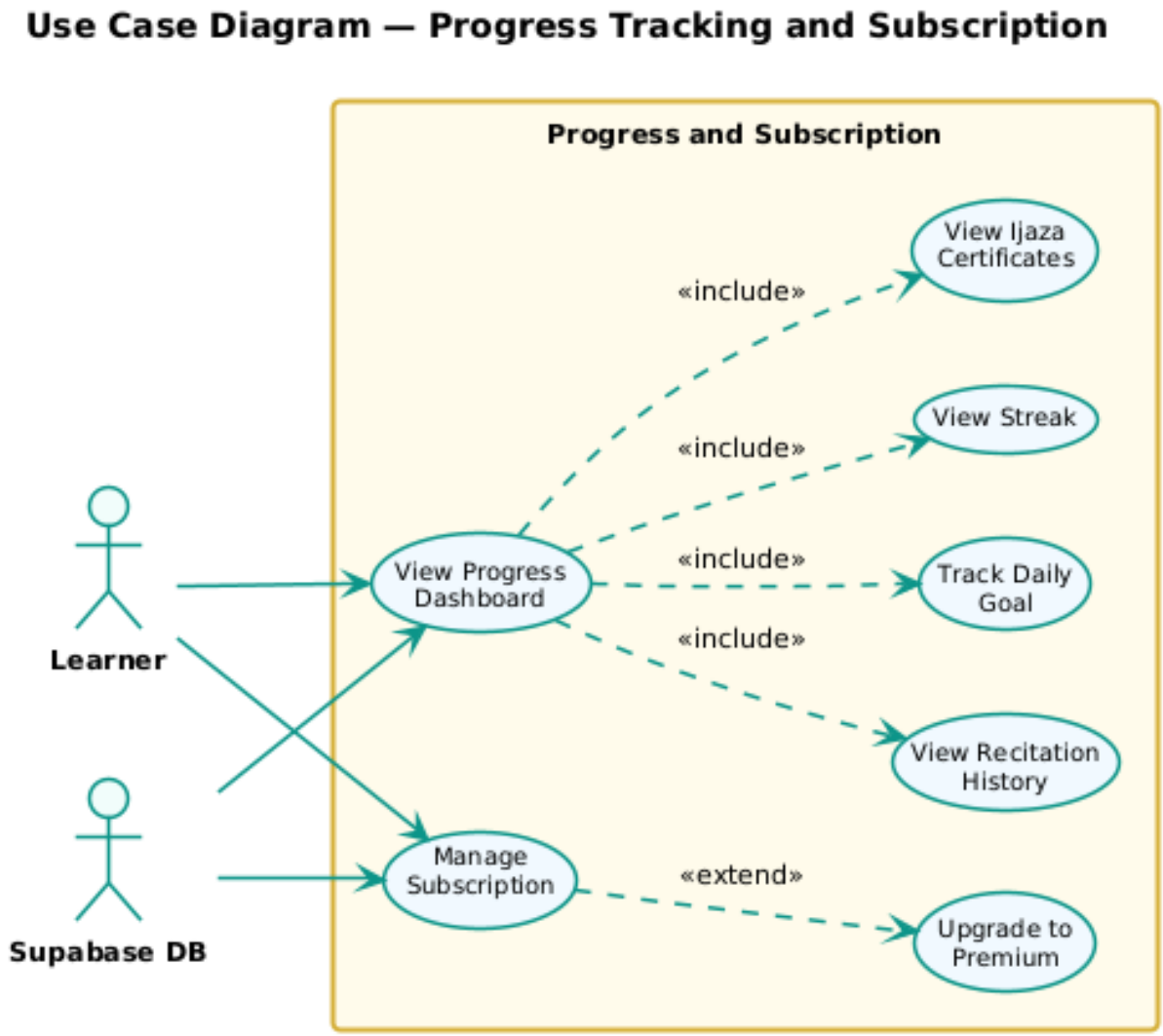


Figure 2.5: Use case diagram for Progress Tracking and Subscription

2.5 Sequence Diagrams

Sequence Diagram

UML sequence diagrams model the interactions and behaviors of system components over time. They visually represent how objects communicate through messages, making them valuable for requirements clarification and system design.

2.5.1 New Actors and Services Identification

Table 2.7: Actors of sequence diagrams

Actor	**Role**
Tahkik App (Flutter)	The mobile client application used by learners.
Tahkik Dashboard (React)	The web dashboard used by teachers and admins.
Supabase Auth	Authentication service handling JWT tokens and session management.
Supabase DB	PostgreSQL database storing all application data.
Tahkik AI Server	Backend server (Python/Whisper) that processes audio and returns AI feedback.

2.5.2 Sequence Diagram: Login

The following sequence diagram illustrates the login process for a Learner using the Tahkik mobile app.

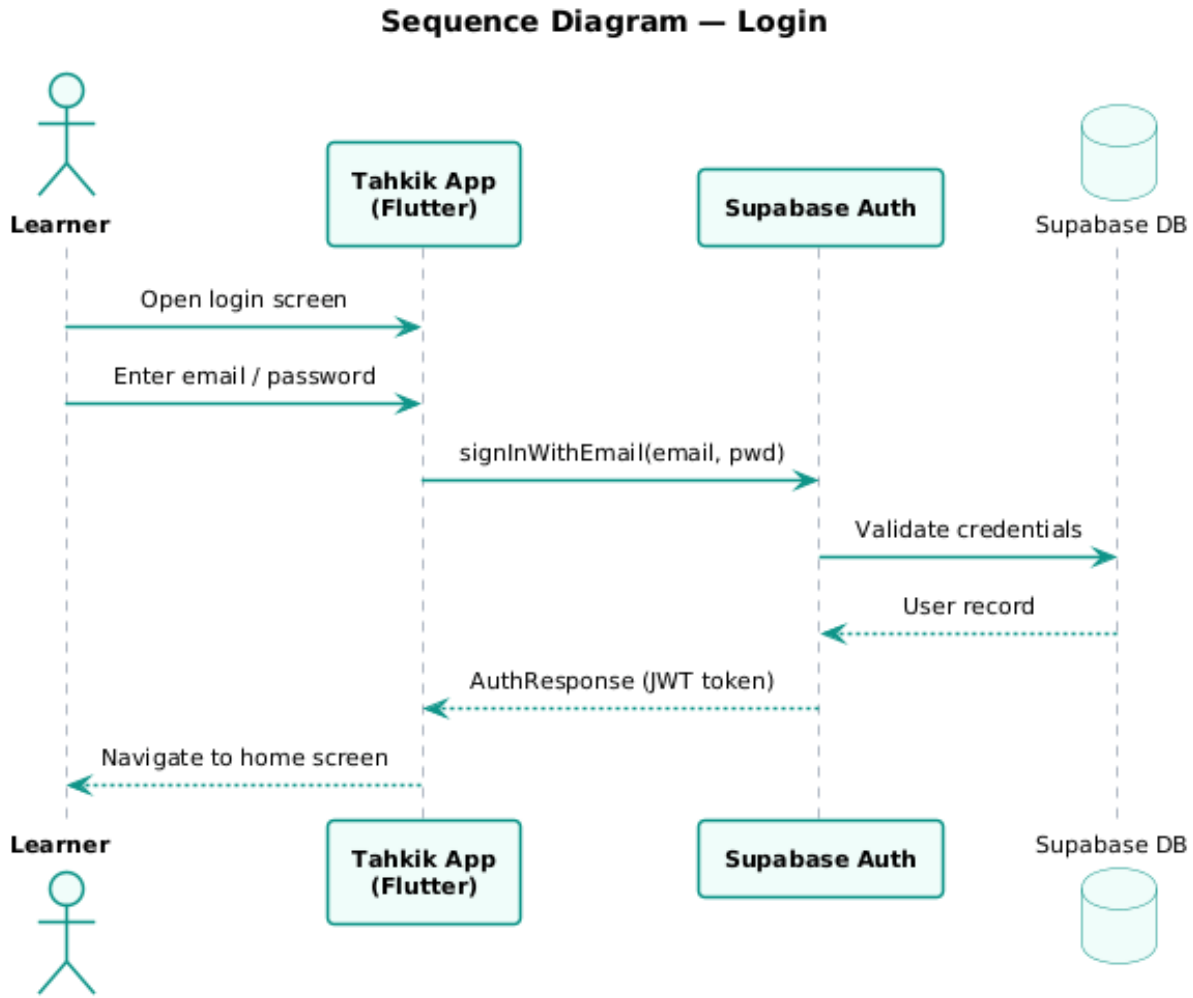


Figure 2.6: Sequence diagram for Login

2.5.3 Sequence Diagram: Sign-up

The following sequence diagram illustrates the sign-up process.

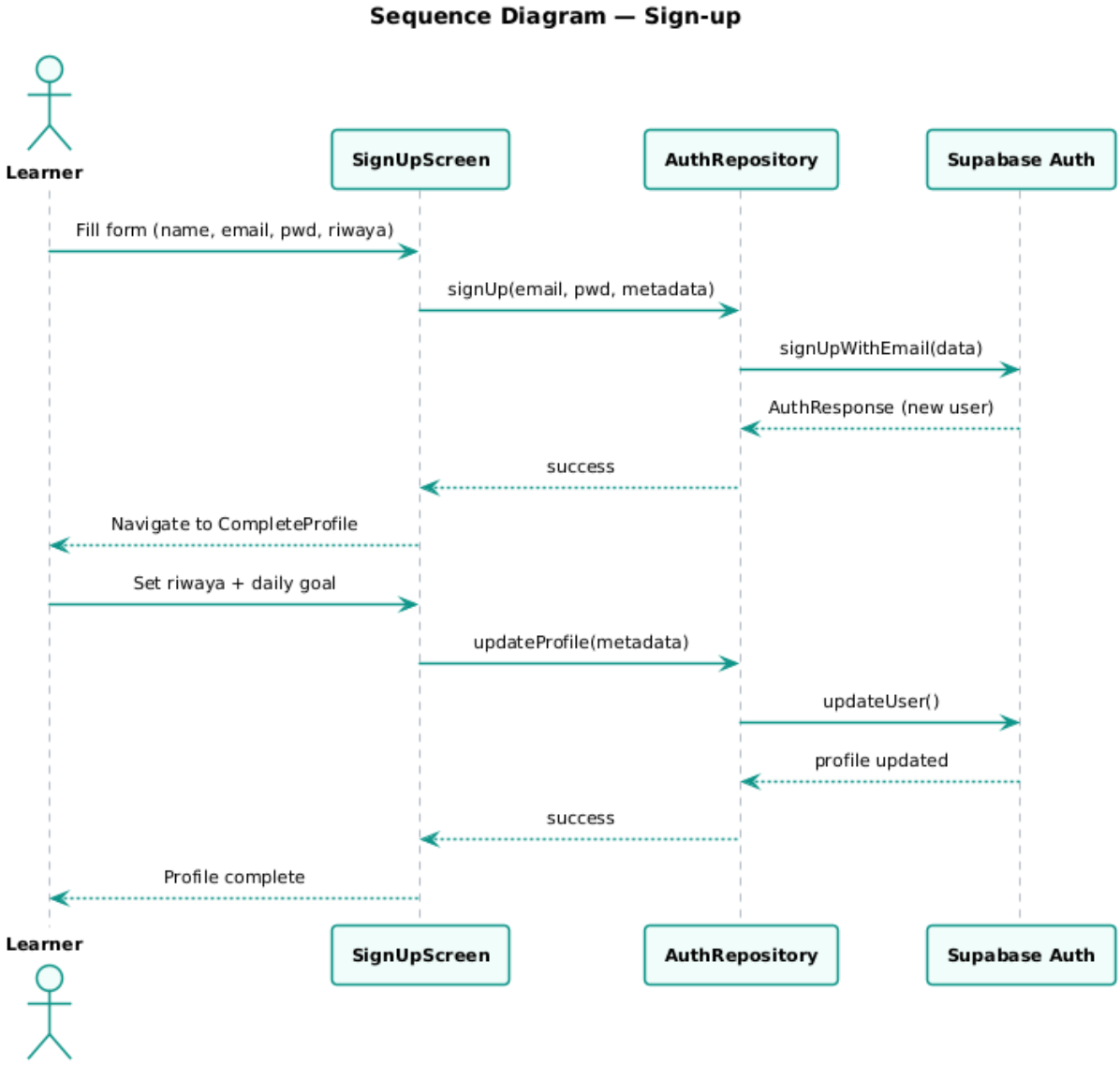


Figure 2.7: Sequence diagram for Sign-up

2.5.4 Sequence Diagram: Recitation Submission and AI Feedback

The following sequence diagram illustrates the core recitation and AI feedback process.

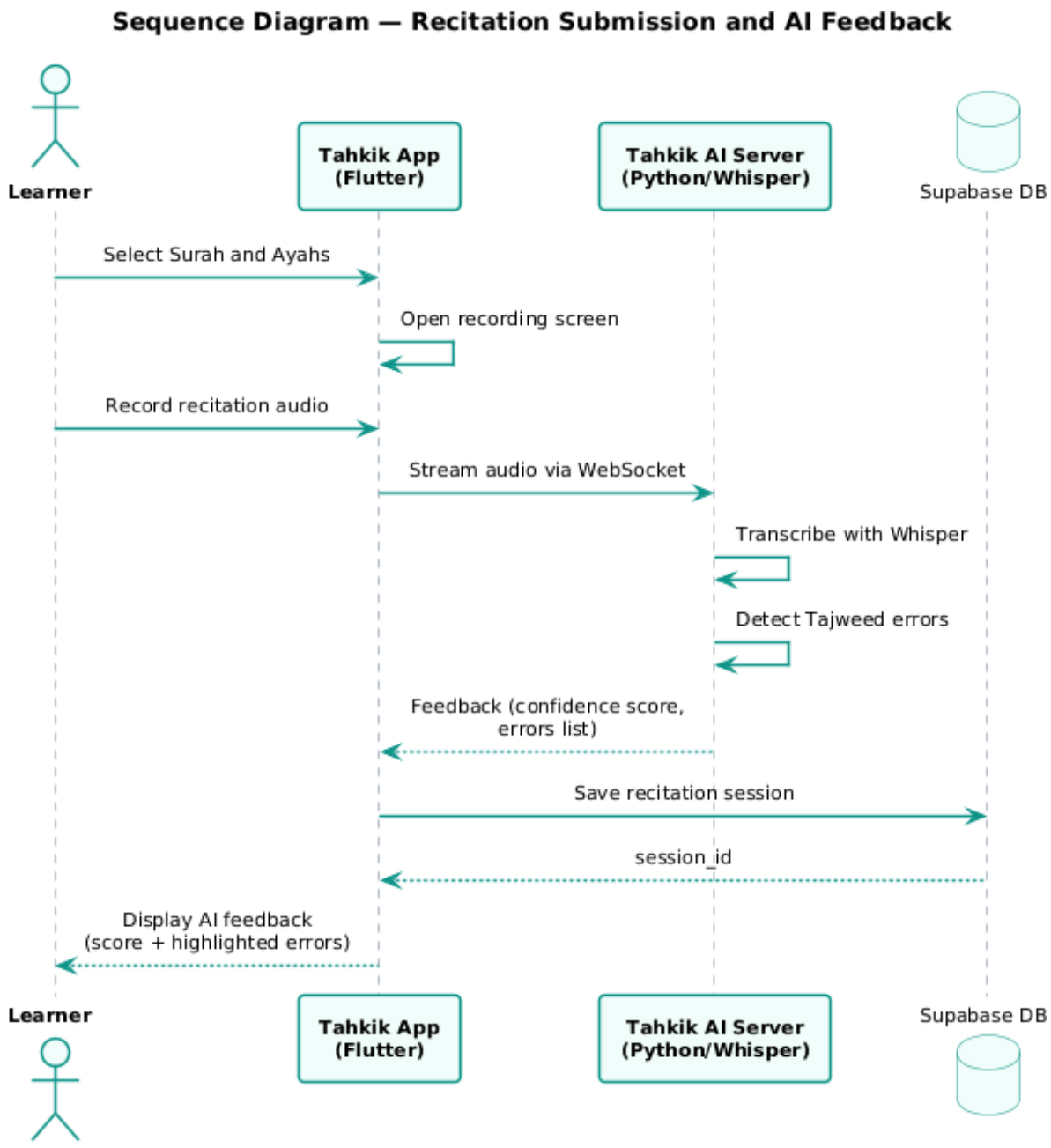


Figure 2.8: Sequence diagram for Recitation and AI Feedback

2.5.5 Sequence Diagram: Teacher Recitation Review

The following sequence diagram illustrates the teacher review process.

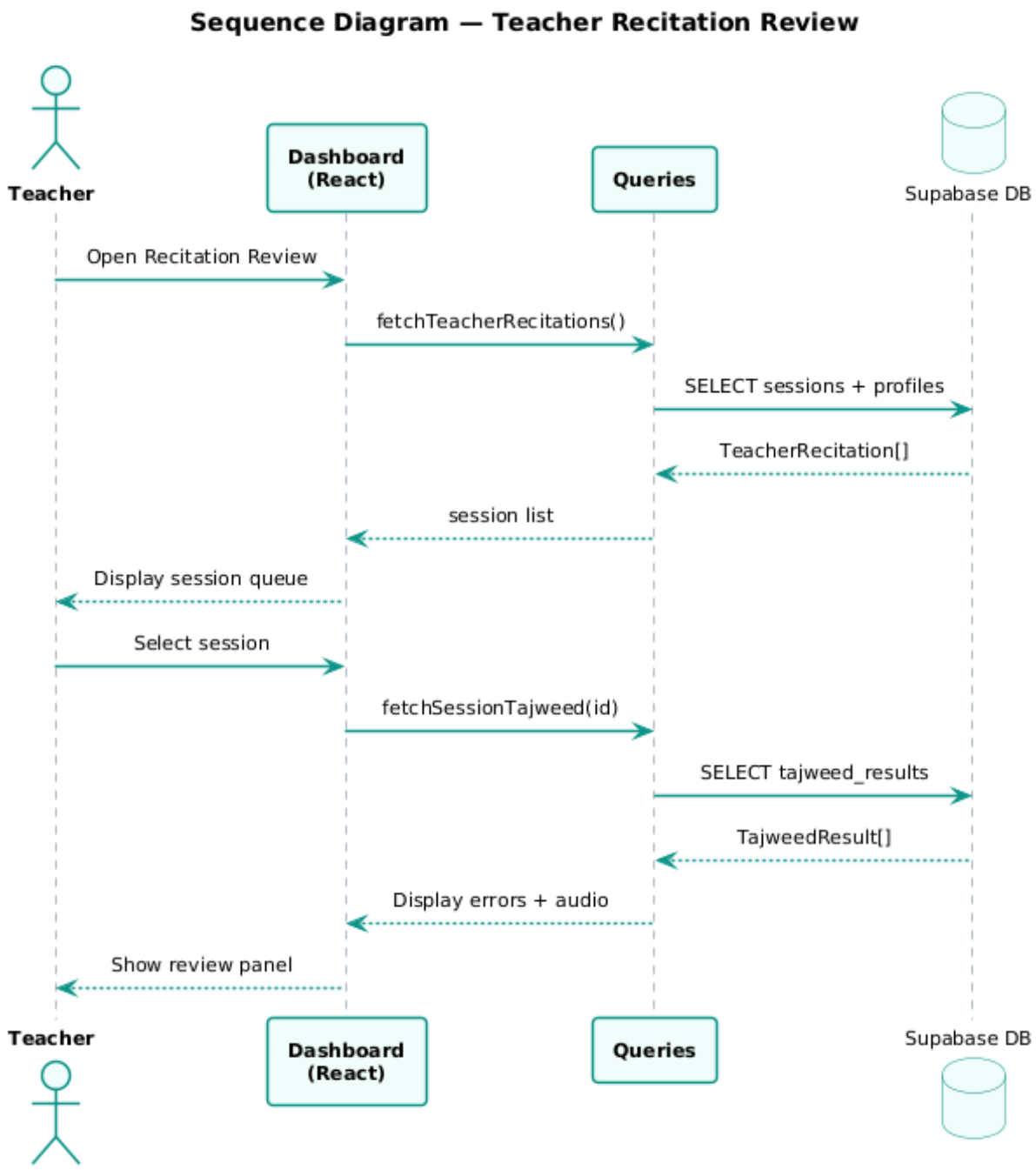


Figure 2.9: Sequence diagram for Teacher Recitation Review

2.5.6 Sequence Diagram: Progress Tracking

The following sequence diagram illustrates the progress tracking process.

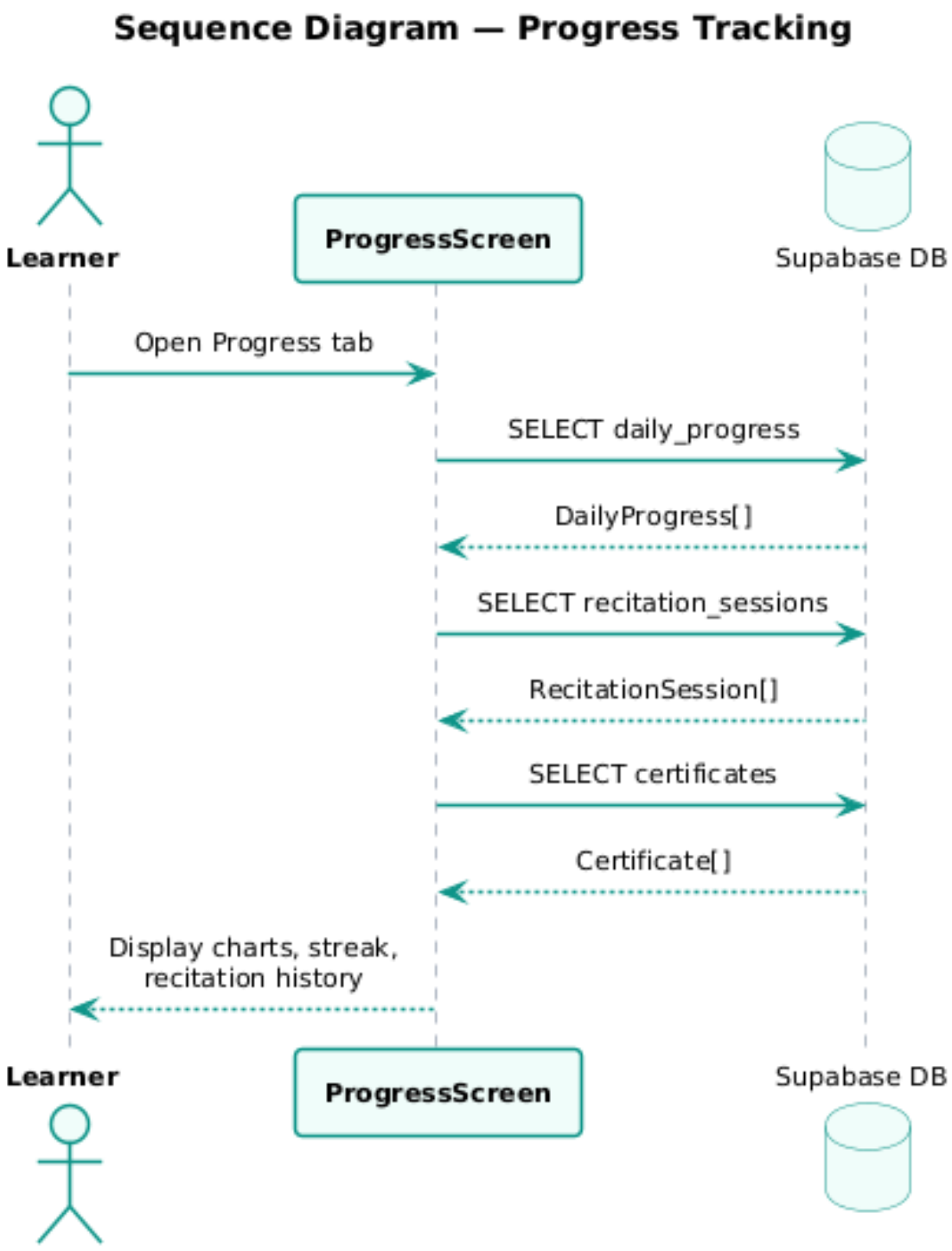


Figure 2.10: Sequence diagram for Progress Tracking

2.5.7 Sequence Diagram: Admin User Management

The following sequence diagram illustrates the admin management process.

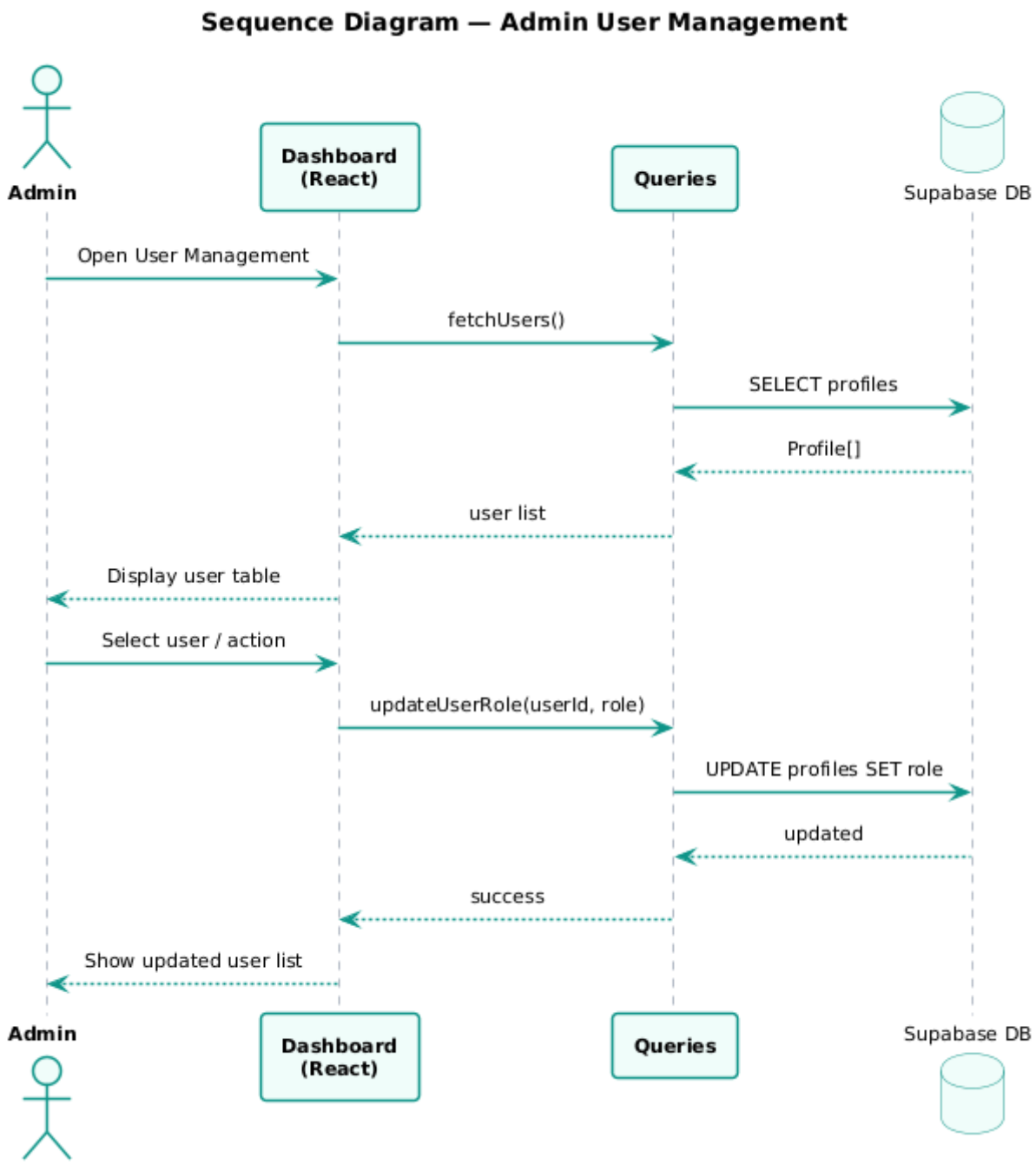


Figure 2.11: Sequence diagram for Admin User Management

2.6 Class Diagram

Class Diagram

A class diagram is a crucial UML diagram representing both static and behavioral structures of an object-oriented system. It shows the classes, their attributes, operations, and the relationships among them.

2.6.1 Class Diagram Presentation

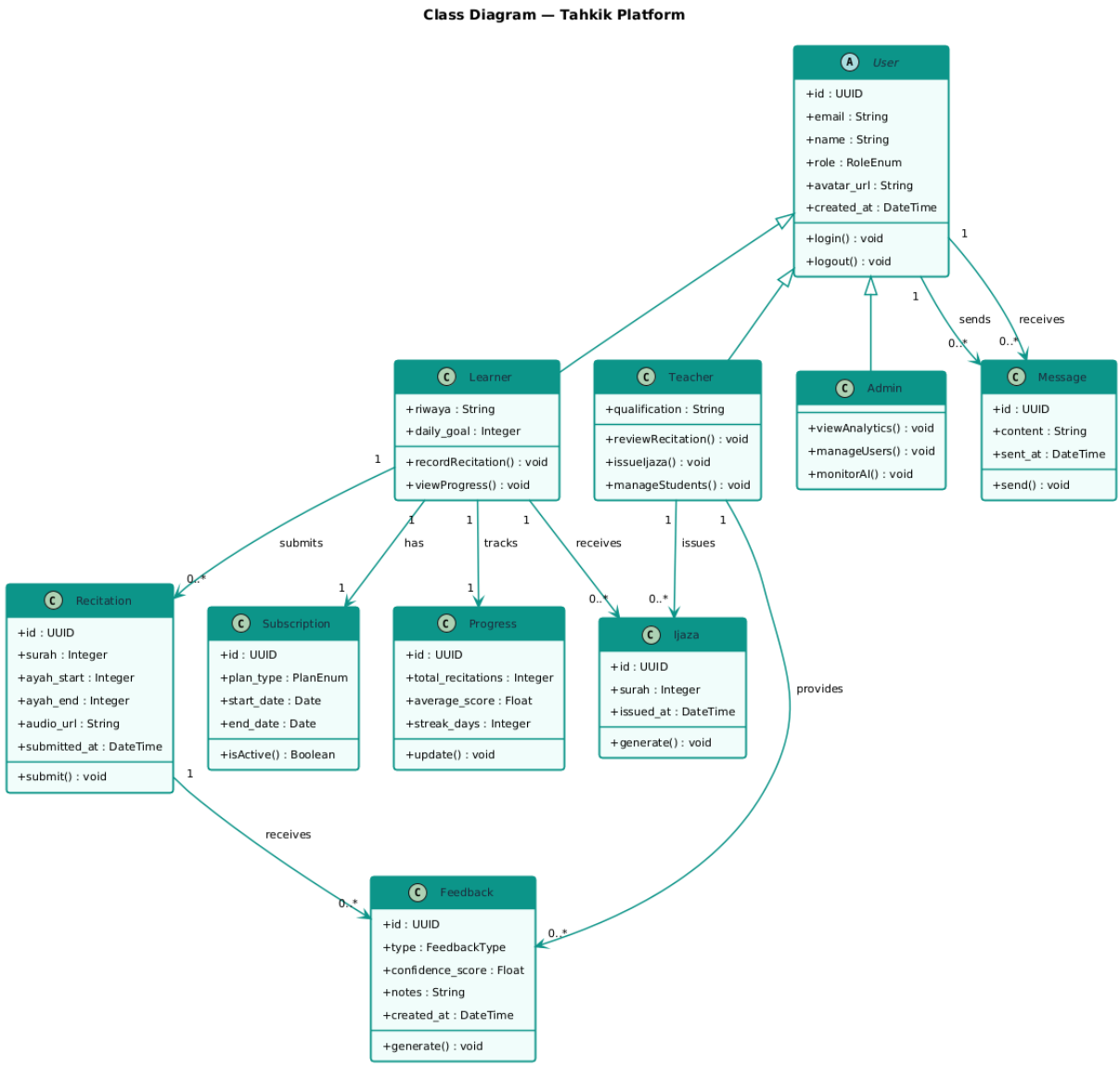


Figure 2.12: Class diagram of the Tahkik platform

2.6.2 Class Diagram Description

Table 2.8: Description of Class diagram entities

Class	Description
User	Base class for all system users with common attributes (id, email, name, role).
Learner	Inherits from User. Represents learners who practice recitation and track progress.
Teacher	Inherits from User. Represents teachers who review recitations and manage students.
Admin	Inherits from User. Represents administrators who manage the platform.
Recitation	A recorded recitation submission with surah, ayah range, and audio URL.
Feedback	Feedback on a recitation, either AI-generated or manual (from teacher).
Subscription	Tracks user subscription plans (free, premium) with start and end dates.
Progress	Aggregates a learner's performance data over time.
Ijaza	A digital certification issued by a teacher to a qualifying learner.
Message	Messages exchanged between teachers and learners.

2.7 Database Design

2.7.1 Relational Schema

The relational model was derived from the UML class diagram by mapping each class into a relation:

Relational Schema

User(<u>id</u>, email, name, role, avatar_url, created_at)
Recitation(<u>id</u>, *user_id*, surah, ayah_start, ayah_end, audio_url, submitted_at)
Feedback(<u>id</u>, *recitation_id*, *teacher_id*, type, confidence_score, notes, created_at)
Subscription(<u>id</u>, *user_id*, plan_type, start_date, end_date)

Progress(id, *user_id*, total_recitations, average_score, streak_days)
Ijaza(id, *learner_id*, *teacher_id*, surah, issued_at)
Message(id, *sender_id*, *receiver_id*, content, sent_at)

Where underlined attributes are primary keys and *italicized* attributes are foreign keys.

2.8 System Architecture

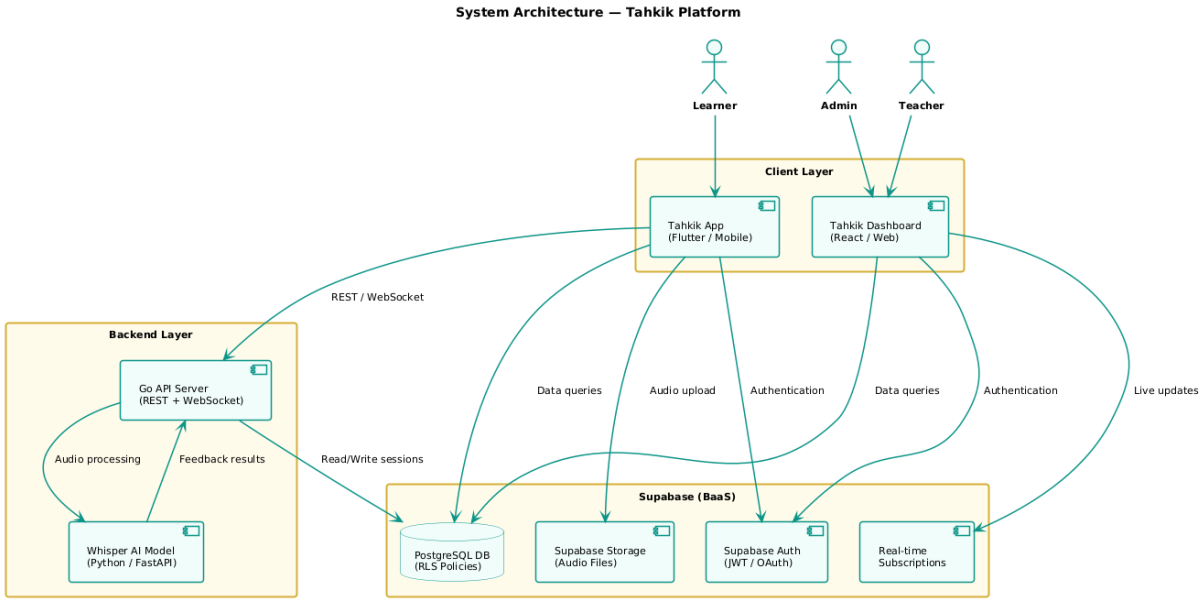


Figure 2.13: System architecture diagram

2.9 Conclusion

This chapter presented the conceptual foundation of the Tahkik platform. We identified 10 functional and 5 non-functional requirements, designed the system's behavioral interactions through 5 use case diagrams and 6 sequence diagrams, and defined the static structure via a class diagram and relational schema. The next chapter details the implementation and realization of this design.

3. Implementation

3.1 Introduction

This chapter presents the technical implementation of the Tahkik platform. We begin by introducing the languages and frameworks used across all system components, then walk through the realised interfaces of both the mobile application (*Tahkik App*) and the web dashboard (*Tahkik Dashboard*).

Scope of this Chapter

Two specialised components — the **Go/Python inference server** and the **fine-tuned Whisper model** — are described at a high level here and documented in depth in two companion technical reports that are appended to this dissertation:

- *Tahkik Inference Server — Technical Report* (Go API gateway, FastAPI inference backend, faster-whisper / CTranslate2 runtime, WebSocket streaming protocol, Hugging Face Space deployment, performance & security).
- *AI-Powered Quranic Recitation Analysis — Implementation Report* (Warsh dataset preparation, data quality assurance pipeline, Whisper fine-tuning runs, WER results, model publication on Hugging Face Hub).

This chapter therefore concentrates on the two client-side components that are not covered elsewhere: the Flutter mobile application and the React web dashboard.

3.2 Technologies and Frameworks

Technology Stack

The Tahkik platform is built on a modern, cross-platform stack that separates concerns across the mobile client, web dashboard, AI backend, and data layer. Each technology was chosen for its performance, community support, and fitness for the specific layer it serves.

3.2.1 Mobile Application — Flutter & Dart

Figure 3.1: Flutter and Dart logos

Flutter is Google's open-source UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase. The Tahkik mobile application targets Android and iOS learners and is built entirely in Flutter, enabling consistent UI and shared business logic across both platforms.

Dart is the programming language underlying Flutter. Its strong typing, async/await model, and just-in-time compilation make it well suited for reactive UI development and real-time audio streaming.

3.2.2 Web Dashboard — React

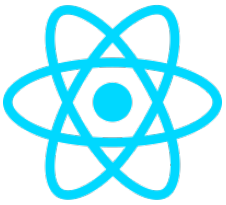


Figure 3.2: React logo

React is a declarative, component-based JavaScript library developed by Meta for building user interfaces. The Tahkik Dashboard — used by teachers and administrators — is a React single-page application that communicates with Supabase directly through its client SDK and with the Go server for AI-related operations.

3.2.3 Backend API — Go

Figure 3.3: Go logo

Go (Golang) is a statically typed, compiled language developed by Google, known for its simplicity and high concurrency performance. The Tahkik Go server acts as a lightweight REST gateway in front of the Python inference backend: it accepts batch audio uploads on `POST /api/v1/evaluate`, validates them, forwards them to the inference service, and exposes a `GET /health` endpoint that the web dashboard polls for live system telemetry. The complementary real-time WebSocket streaming path (`/ws/stream`) is served by the FastAPI inference backend directly to the mobile client; the Go server is intentionally kept stateless and proxy-free for streaming workloads. The full server architecture, request lifecycle, and deployment procedure are detailed in the companion *Tahkik Inference Server* report.

3.2.4 AI Server — Python & FastAPI

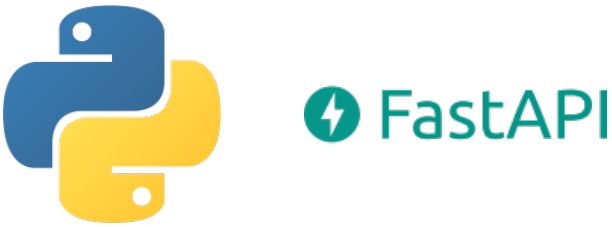


Figure 3.4: Python and FastAPI logos

Python is the leading language for machine learning and audio processing. The AI backend runs a fine-tuned variant of OpenAI's **Whisper** model, specialised on Warsh *'an Nafi'* recitation and published on the Hugging Face Hub as `benhadjermed/tahkik-small-warsh` (with a `tahkik-basic-warsh` predecessor). The production deployment uses `faster-whisper` (CTranslate2 with INT8 quantisation) on a Hugging Face Space for sub-second inference, while a PyTorch-based local server is retained for development. The full fine-tuning pipeline — Warsh dataset alignment, quality-assurance scripts, multi-reciter training runs

and WER analysis — is documented in the companion *AI-Powered Quranic Recitation Analysis* report.

FastAPI is a modern, high-performance Python web framework for building APIs with automatic OpenAPI documentation. It serves as the HTTP/WebSocket interface for the Whisper inference pipeline: it exposes `POST /evaluate` for batch audio (called by the Go gateway) and `GET /ws/stream` for the real-time streaming protocol consumed directly by the Flutter mobile client.

3.2.5 Backend-as-a-Service — Supabase

Figure 3.5: Supabase logo

Supabase is an open-source Firebase alternative built on top of PostgreSQL. It provides the Tahkik platform with:

- **Authentication** — JWT-based email/password and Google Sign-In.
- **Database** — PostgreSQL with Row Level Security (RLS) policies.
- **Storage** — Audio file storage for submitted recitations.
- **Real-time** — Live subscriptions for teacher notifications.

Figure 3.6: PostgreSQL — underlying Supabase database engine

3.2.6 Technology Stack Summary

Table 3.1: Summary of technologies used in the Tahkik platform

Layer	**Technology**	**Role**
Mobile App	Flutter / Dart	Cross-platform learner application (iOS & Android)
Web Dashboard	React / TypeScript / Vite	Teacher and admin web interface
Backend API	Go (`net/http`)	REST gateway, batch `/evaluate`, `/health` probe
AI Server	Python / FastAPI	Whisper inference, WebSocket streaming
BaaS	Supabase	Auth (JWT), PostgreSQL + RLS, Storage, Real-time
AI Runtime	faster-whisper / CTranslate2	INT8-quantised Whisper inference engine
AI Model	`tahkik-small-warsh`	Whisper fine-tuned on Warsh recitation
ML Hosting	Hugging Face Spaces (Docker)	Production deployment of the inference server

3.3 Mobile Application — Tahkik App

Mobile Application

The Tahkik mobile application is the primary interface for learners. Built with Flutter targeting Android and iOS, it covers the complete learner journey: an onboarding tour, authentication and profile completion, a Warsh Quran reader, real-time recitation recording with WebSocket-based AI transcription, Tajweed feedback visualisation, progress tracking, account settings, and subscription management.

3.3.1 Application Architecture and Libraries

The mobile application is structured as a feature-first, layered Flutter project. Each feature module (`auth`, `onboarding`, `home`, `quran_reader`, `recitation`, `tajweed_feedback`, `progress`, `settings`, `subscription`) is split into `data/`, `domain/`, `presentation/` and `providers/` sub-folders, while cross-cutting concerns live under `core/` and `shared/`.

Table 3.2: Key libraries used by the Tahkik mobile application

Concern	**Package**	**Role in the app**
State management	`flutter_riverpod`, `riverpod_annotation`	Reactive providers for auth, audio, transcription
Navigation	`go_router`	Declarative routing with redirect-based auth guards
HTTP client	`dio`	REST calls to the Go gateway with JWT Bearer interceptor
Realtime stream	`web_socket_channel`	Streaming PCM audio to the inference WebSocket
Audio I/O	`record`, `audioplayers`	Microphone capture and playback for review
Auth backend	`supabase_flutter`, `google_sign_in`	JWT auth with email/password and Google OAuth
Local storage	`hive_flutter`, `shared_preferences`	Offline cache for verses, preferences, session data
Config	`flutter_dotenv`	Loads `API_BASE_URL`, `HF_SPACE_URL`, Supabase keys
Charts	`fl_chart`	Progress and streak visualisations
Typography & UI	`google_fonts`, `flutter_svg`, `shimmer`, `cached_network_image`	Latin/Arabic fonts, SVG icons, skeletons, image cache
File system	`path_provider`	Temporary storage for recorded audio
Serialisation	`json_annotation`, `json_serializable`	Generated codecs for API DTOs

Custom Warsh Font

The Mushaf reader renders verses using the official **King Fahd Glorious Quran Printing Complex** Warsh font, bundled as the `KFGQPC_Warsh.ttf` asset (declared under the `QuranWarsh` family in `pubspec.yaml`). This guarantees that ligatures, sukoon, tanween, and Warsh-specific orthographic glyphs (e.g. Alef Maqsura variants) render exactly as in a printed Mushaf.

3.3.2 API Layer and Authenticated Requests

The `ApiClient` wraps a single Dio instance configured against `API_BASE_URL` (default `https://api.tahkik.app/v1`). A custom interceptor automatically attaches the current Supabase session as an `Authorization: Bearer <JWT>` header on every request, so feature repositories never have to handle credentials manually. The most relevant REST endpoints consumed by the app are summarised below; the streaming WebSocket (`wss://<hf-space>/ws/stream`) is described in the companion server report.

Table 3.3: REST endpoints consumed by the Flutter client

Group	Endpoint	Purpose
Recitation	`POST /recitation/analyze`	Batch audio evaluati streaming)
Recitation	`GET /recitation/history`	Past submissions for t
Quran	`GET /quran/surahs`	List of Surahs metada
Quran	`GET /quran/surahs/{n}/ayahs`	Ayahs for a given Sur
User	`GET /user/profile`, `GET /user/progress`	Profile and aggregate
Subscription	`GET /subscription/plans`, `POST /subscription/subscribe`	Tiers and purchase

3.3.3 Navigation Shell

A `ShellRoute` hosts the four primary tabs of the application, with the bottom navigation bar (`TahkikBottomNav`) reacting to the current GoRouter location.

Table 3.4: Bottom navigation tabs of the Tahkik app

Route	Tab	Purpose
`/`	Home	Daily verse, streak, quick practice entry
`/practice`	Practice	Surah/ayah selection and live recitation
`/progress`	Progress	Charts, streaks, recent sessions, scores
`/settings`	Settings	Account, display, riwaya, daily goal

The router's `redirect` hook enforces the full auth flow: unauthenticated users are sent to `/onboarding`; authenticated users with an incomplete profile are routed to `/complete-profile`; only fully-onboarded users reach the shell routes above. The redirect listens to `authStateProvider` so it re-evaluates on every Supabase auth event.

3.3.4 Onboarding Flow

First-time visitors land on the **Get Started** screen which introduces the platform and gates entry to the rest of the funnel. A second **AI Feedback Onboarding** screen explains how recitation is recorded, transcribed by the AI, and reviewed by a teacher — setting accurate expectations about real-time partial transcription and confidence scoring.

Screenshot: Get Started — intro to the Tahkik platform

Screenshot: AI Feedback onboarding — how live transcription and teacher review work

3.3.5 Authentication Screens

The authentication flow combines Supabase email/password and Google Sign-In (via the `google_sign_in` package). A dedicated **Create Account** screen handles registration with form validation, while the **Complete Profile** screen captures the three fields required by the rest of the app:

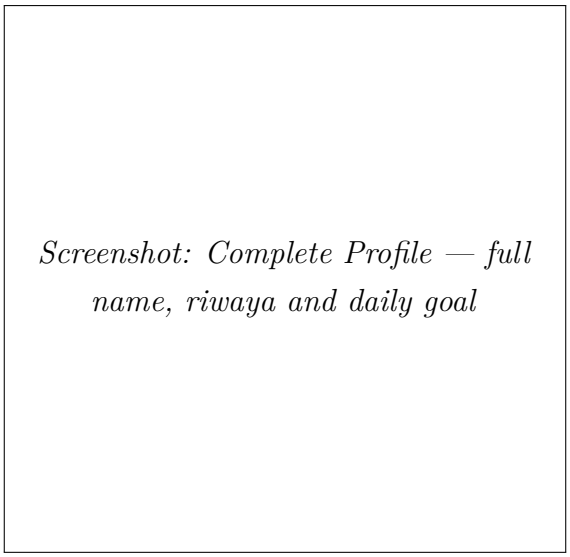
- `full_name` — displayed across the UI and on Ijaza certificates

- `preferred_riwaya` — *Warsh* or *Hafs*, persisted in the user metadata
- `daily_goal_minutes` — drives the streak/goal logic on the Home and Progress screens

The `profileCompleteProvider` reads these fields from `user.userMetadata` and re-evaluates after every `AuthChangeEvent.userUpdated`, ensuring the router immediately redirects out of `/complete-profile` once the form is submitted.

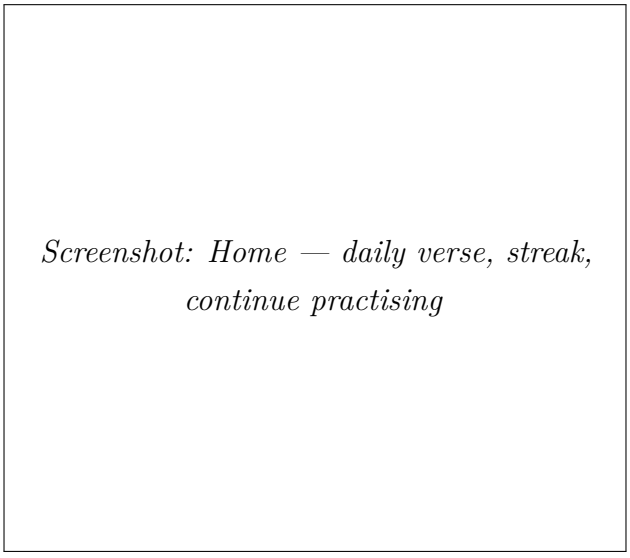
Screenshot: Login — email/password and Google Sign-In

Screenshot: Sign-up — account creation



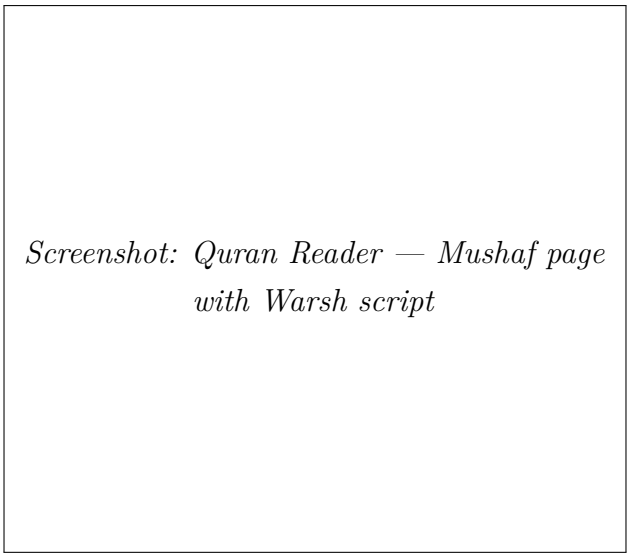
3.3.6 Home Screen

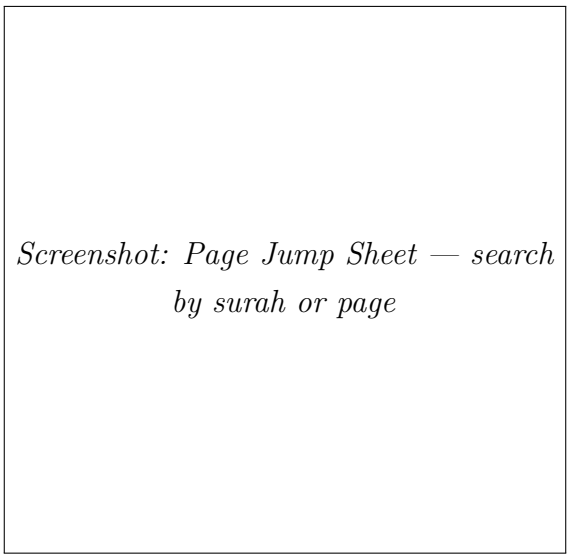
The Home screen is the daily landing page. It surfaces the curated **Daily Verse** (managed from the admin dashboard's Daily Verse module), the user's current streak and daily-goal progress ring, a "Continue practising" shortcut to the last surah, and a vertical scroll of recently-recited ayahs with their AI scores.



3.3.7 Quran Reader — Mushaf Page View

The Mushaf Page View renders the Quran one Mushaf page at a time using the custom Warsh font. A `PageView` swipes between the 604 standard Mushaf pages, and a **Page Jump Sheet** lets the user jump directly to a surah or page number via a searchable bottom sheet. Verse boundaries are tappable: long-press selects a range, single-tap opens a context menu with *Practise this verse*, *Listen*, and *Bookmark*.





3.3.8 Practice Screen and Tajweed Feedback

The Practice screen is the core feature of the app. The user selects a verse range, then a large **Microphone FAB** (`microphone_fab.dart`) starts recording with the `record` package at 16 kHz PCM. A live **Waveform Visualiser** animates while audio chunks are streamed over WebSocket to the inference service. Partial transcriptions arrive every $\sim$1 s and are overlaid in real time on the corresponding **Ayah Card**. A **Practice Theme Selector** lets the user switch between calm, focus and traditional colour palettes.

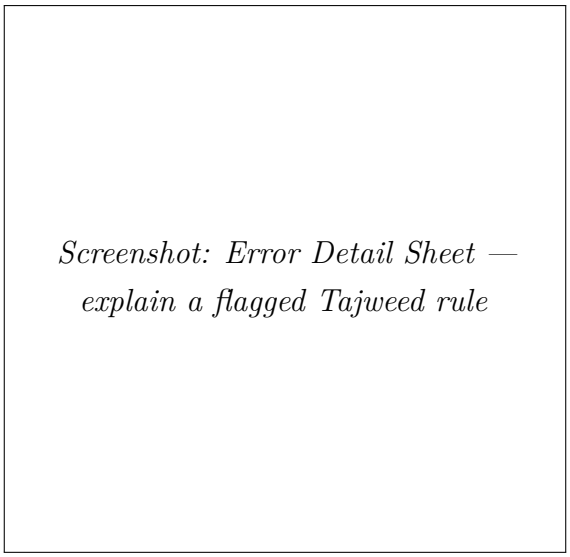
When recording stops, the final result — transcription, confidence score, and per-word Tajweed annotations — is rendered by the `tajweed_feedback` module:

- **Overall Score widget**: animated radial gauge showing the confidence percentage.
- **Tajweed Overlay**: highlights each word with a colour reflecting the detected error category (madd, ghunnah, qalqalah, etc.).

- **Error Detail Sheet**: a bottom sheet that, when a highlighted word is tapped, explains the rule, plays the correct pronunciation, and proposes a corrective exercise.

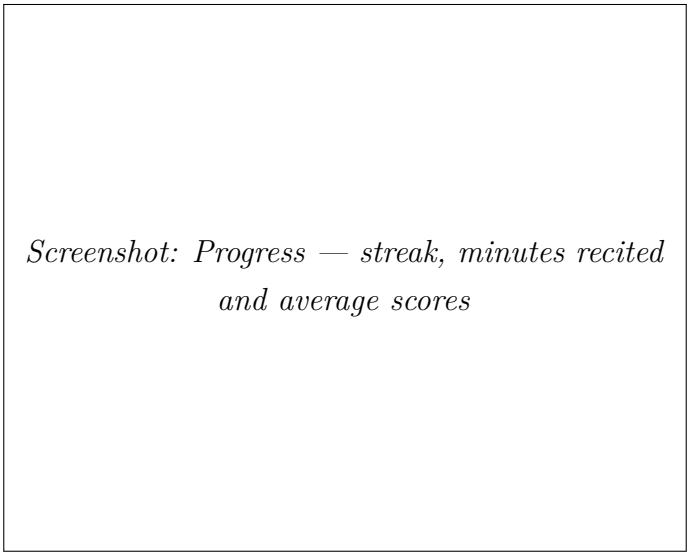
Screenshot: Practice — ayah card, microphone FAB and waveform

Screenshot: Tajweed feedback — score gauge and per-word overlay



3.3.9 Progress Screen

The Progress screen aggregates the learner's data via the `/user/progress` endpoint and renders it with `fl_chart`: a 30-day line chart of minutes recited, a bar chart of average scores per surah, a streak calendar, and a recent-sessions list. Each session row links back to the corresponding Practice result so the user can replay the audio and re-read the Tajweed feedback.

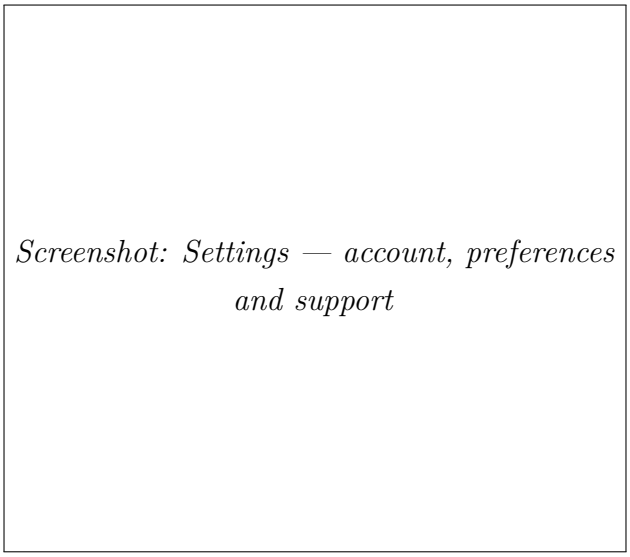


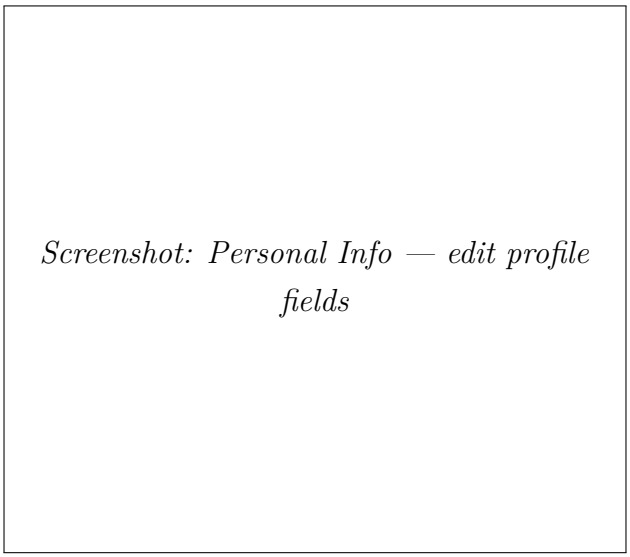
3.3.10 Settings, Personal Info and Display Settings

The Settings tab is the user's control centre. It groups account, preference and support items, and opens dedicated screens for the most detailed configuration:

- **Personal Info screen** — edit full name, email, avatar, and update Supabase user metadata.

- **Display Settings modal** — font size, line height, day/night theme, optional tashkeel display, and reciter audio preference for verse playback. Preferences are persisted via `shared_preferences` so they survive cold starts.
- **Riwaya selector** — switch between Warsh and Hafs; updates `preferred_riwaya` in the user metadata.
- **Daily goal selector** — adjusts the target minutes per day used by the Home streak ring.





Screenshot: Display Settings modal — font, theme and recitation preferences

3.3.11 Subscription — Choose Plan

A dedicated **Choose Plan** screen (`choose_plan_screen.dart`, reached via the `/plan` route) compares the Free and Premium tiers side by side: number of AI feedbacks per day, access to a personal teacher, ability to receive an Ijaza certificate, and access to advanced analytics. Plan metadata is fetched from `/subscription/plans` and purchases are submitted via `/subscription/subscribe`.

Screenshot: Choose Plan — Free vs Premium comparison

3.4 Web Dashboard — Tahkik Dashboard

Web Dashboard

The Tahkik Dashboard is a React 18 + TypeScript single-page application built with Vite, React Router v6 (data router), TanStack Query v5 for server state, and Recharts for visualisations. It is the unified entry point for both administrators and teachers — role-based routing redirects each authenticated user to the appropriate workspace based on their record in the `admins` table. The dashboard talks directly to Supabase for authentication, database reads/writes, and real-time subscriptions, and consults the Tahkik Go server (`VITE_TAHKIK_SERVER_URL`) for live inference telemetry. Styling is built on CSS Modules with a tokenised light/dark theme, and the test suite uses Vitest + React Testing Library.

3.4.1 Application Architecture and Libraries

The dashboard follows a feature-first folder layout under `src/`: a small `app/` shell wires the router, providers, an `ErrorBoundary`, and a `RequireAuth` guard; cross-feature primitives live in `components/`; per-domain pages live under `features/auth`, `features/admin`, and `features/teacher`; and the data layer is split between `lib/queries/` (read hooks) and `lib/mutations/` (write hooks). Theming tokens are defined once under `theme/tokens.css` and consumed everywhere via CSS Modules.

Table 3.5: Key libraries used by the Tahkik web dashboard

Concern	**Package**	**Role in the dashboard**
Build / dev server	`vite`, `typescript`	Fast HMR, type-checked production build
UI framework	`react`, `react-dom`	Component-based UI
Routing	`react-router-dom` (v6 data router)	Role-based layouts, nested routes, loaders
Server state	`@tanstack/react-query` (v5)	Cached reads, optimistic mutations, real-time invalidation
Auth & DB	`@supabase/supabase-js`	JWT auth, PostgreSQL with RLS, real-time channels
Charts	`recharts`	Line, donut and bar charts
Toasts	`sonner`	Non-blocking notifications and alert strip
Styling	CSS Modules + `theme/tokens.css`	Scoped styles, light/dark theme variables
Testing	`vitest`, `@testing-library/react`, `jsdom`	Unit and component tests
Tooling	`eslint`, `prettier`	Linting and formatting

3.4.2 Application Shell, Auth Guard and Error Boundary

The router is built with React Router's *data router* API. A top-level `ErrorBoundary` catches rendering and loader exceptions and surfaces a friendly retry screen, while `RequireAuth` wraps every protected route, subscribing to `AuthProvider` to enforce the following invariants:

- Unauthenticated visitors are redirected to `/login`.
- Authenticated users without a matching row in the `admins` table are sent to `/unauthorized`.
- Visiting `/` redirects an admin to `/admin` and a teacher to `/teacher`, based on the `role` column.

`AuthProvider` listens to `supabase.auth.onAuthStateChange` and exposes `{ user, role, isLoading }` to the rest of the tree via React context, so every page consumes a single

source of truth for authentication.

3.4.3 Styling and Light/Dark Theme

All visual tokens (colours, spacing, radii, typography, shadow, transition durations) are declared as CSS custom properties in `theme/tokens.css`. A `ThemeProvider` toggles a `data-theme` attribute on the document root between `light` and `dark`, and components consume the tokens through scoped CSS Modules. Skeleton loaders (`Skeleton`, `SkeletonPage`, `SkeletonShell`) reuse the same tokens so the loading state is visually consistent across pages.

3.4.4 Shared Component Library

A small but consistent set of primitives is reused across both workspaces:

Table 3.6: Shared UI components

Component	Purpose
`Card`, `StatCard`	Containers for grouped information tiles
`Button`, `Input`, `Badge`	Form controls and pill-shaped status
`Avatar`	User avatar with fallback initial
`LineChart`, `DonutChart`	Thin wrappers around Recharts
`ProgressBar`	Linear progress with token-driven col
`Sidebar`, `Topbar`	Layout chrome shared by Admin and
`Skeleton`, `SkeletonPage`, `SkeletonShell`, `LoadingSkeleton`	Granular loading placeholders

3.4.5 Data Layer — TanStack Query Hooks

Every page is hydrated through TanStack Query hooks split by domain:

- `lib/queries/admin.ts`: `useAdminOverview`, `useAdminUsers`, `useAdminTeachers`, `useAdminSubscriptions`, `useAdminAnalytics`, etc.
- `lib/queries/teacher.ts`: `useTeacherOverview`, `useTeacherStudents`, `useStudentDetail`, `useRecitationQueue`.
- `lib/queries/messages.ts` and `useMessagesRealtime`: backed by a Supabase real-time channel for in-app chat.

- `lib/queries/certificates.ts` and `lib/queries/daily-verses.ts`: read hooks for the Ijaza and Daily Verse modules.
- `lib/mutations/`: write counterparts for teacher assignment, certificate issuance, and message sending, all using TanStack Query's `onSuccess` cache invalidation to keep the UI in sync.

3.4.6 Environment Configuration

The dashboard reads its configuration from Vite environment variables (`import.meta.env`); a sample is provided as `.env.example`.

Table 3.7: Dashboard environment variables

Variable	Purpose
`VITE_SUPABASE_URL`	Project URL for the Supabase client
`VITE_SUPABASE_ANON_KEY`	Anonymous JWT signing key
`VITE_TAHKIK_SERVER_URL`	Base URL of the Go inference gateway (live AI & System Health card); when empty the page falls back to Supabase-only counters
`VITE_TAHKIK_HEALTH_TOKEN`	Optional Bearer token for the `/health` endpoint of the Go server

3.4.7 Authentication — Login Screen

The login screen offers two authentication paths: classic email/password and Google OAuth Sign-In. Both flows are handled by the Supabase Auth SDK (`signInWithPassword`, `signInWithOAuth`). After successful authentication, the `AuthProvider` verifies the user's role against the `admins` table and routes them to either `/admin` or `/teacher`. Users without a record land on `/unauthorized`.

Screenshot: Login screen with email/password and Google Sign-In

3.4.8 Admin Workspace

The administrator workspace exposes seven pages: Overview, User Management, Teachers, Subscriptions, AI & System Health, Analytics, and Daily Verse. All data is fetched live from Supabase via TanStack Query hooks (`useAdminOverview`, `useAdminUsers`, `useAdminTeachers`, etc.), and the Tahkik Go server is polled for real-time inference telemetry.

Admin Overview

The Overview page summarises the platform at a glance with four key statistics — Total Users, Premium Subscriptions, Minutes Recited Today, and Inference Server status — followed by a 30-day usage trend, a tier-distribution donut chart, and the most recent recitation submissions. The Go-server status indicator turns green when the `/health` endpoint responds with `ok`.

Screenshot: Admin Overview — platform statistics and 30-day usage trend

User Management

Lists all registered accounts with their display name, email, role, subscription plan, join date, and status. A search box filters the list client-side, and the table is fed by the `useAdminUsers` TanStack Query hook reading directly from Supabase with the appropriate RLS policies.

Screenshot: Admin User Management — searchable user list

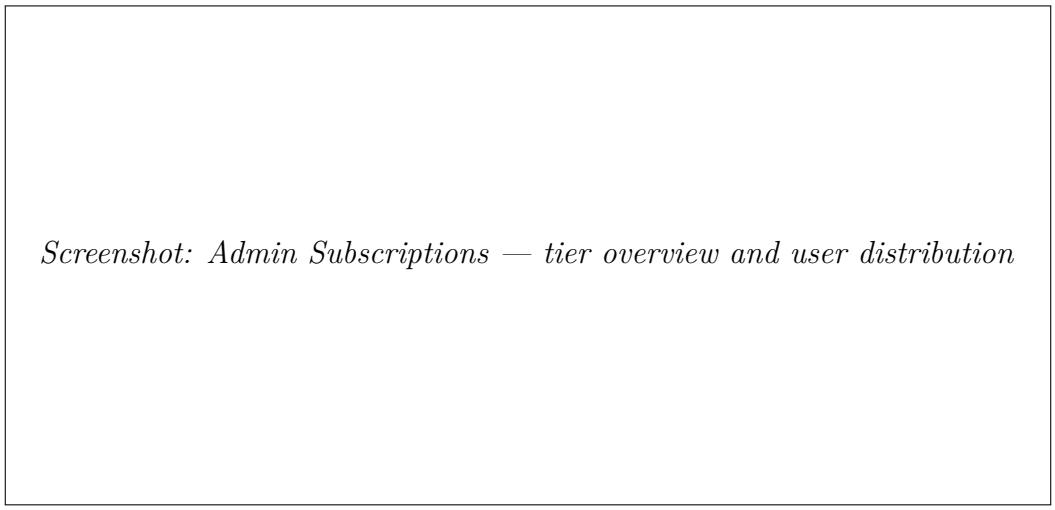
Teachers Management

The Teachers page lets administrators promote existing users to teachers, view each teacher's roster, assign or unassign students, and demote teachers when needed. The "Add Teacher" button opens a dialog (`AddTeacherDialog`) that creates the `admins` record in Supabase and immediately reflects in the list via TanStack Query cache invalidation.

Screenshot: Admin Teachers — manage teacher accounts and student assignments

Subscriptions

Displays the two subscription tiers (Free and Premium) with the number of users on each, the feature breakdown (e.g. AI feedback limits, teacher pairing, Ijaza certificates), and the monthly price. The counts come from the live Supabase profile data via `useAdminSubscriptions`.



AI & System Health

This is the live operations console for the Tahkik AI pipeline. The dashboard polls the Go server's health endpoint and renders:

- **Host metrics** — CPU, memory, disk, and swap utilisation with critical/warning thresholds.
- **Inference latency** — partial and final p95 latency in milliseconds.
- **Request stats** — total and failed request counts with derived error-rate alerts.
- **Container health** — Docker container status (running, unhealthy, exited).
- **Live alert strip** — automatically raises a Sonner toast when a metric crosses the critical threshold.

When `VITE_TAHKIK_SERVER_URL` is unset, the page gracefully falls back to Supabase-only counters so the dashboard remains usable in environments without the inference server.

Screenshot: Admin AI & System Health — live telemetry from the Tahkik Go server

Analytics

The Analytics page summarises platform-wide engagement: total minutes recited (all time), certificates issued, active teachers, and the most-practiced surah. A 30-day Usage Trend line chart, rendered with Recharts, plots minutes recited across all users.

Screenshot: Admin Analytics — usage trend and engagement metrics

Daily Verse

Allows administrators to curate the daily verse displayed on the mobile home screen. A form accepts the Arabic text, English translation, surah name, and ayah number; submitting the form creates the verse and optionally promotes it to active. The page also lists previously published verses, allowing one-click reactivation or deletion.

Screenshot: Admin Daily Verse — manage the rotating daily ayah

3.4.9 Teacher Workspace

The teacher workspace contains five pages: Overview, My Students, Recitation Review, Ijaza Certificates, and Messages. Each teacher only sees data scoped to their assigned students through Supabase Row Level Security.

Teacher Overview

Welcomes the teacher with an Arabic greeting and shows four stat cards: Assigned Students, Recent Sessions (last 7 days), Certificates Issued, and Average Score across all students. Below the stats, a feed lists the most recent recitations submitted for review.

Screenshot: Teacher Overview — daily snapshot of the teacher's roster

My Students and Student Detail

The **My Students** page renders the teacher's full roster: each card shows the student's avatar, name, last-active timestamp, current streak, and a small badge for their average score. Clicking a student navigates to a dedicated **Student Detail** route

(`teacher/StudentDetail.tsx`, hydrated by the `useStudentDetail` query), which expands the snapshot into a full profile: last surah practised, average score, streak history, the most recent recitations queued for review, and a progress bar against the daily goal.

Screenshot: Teacher My Students — roster of assigned students

Screenshot: Student Detail — per-student profile and recent activity

Recitation Review

The core teaching tool. The left pane lists every submission queued for review; selecting one loads the audio player, the AI transcription, the confidence score, and the list of detected Tajweed annotations. The teacher can play/pause audio, scrub the progress bar, and override the AI verdict on individual annotations.

Screenshot: Teacher Recitation Review — listen, inspect AI feedback, override

Ijaza Certificates and Printable View

Teachers can issue digital Ijaza certificates to qualifying students. An **Issue** form accepts the student, the surah, the riwaya (Warsh / Hafs), and optional notes; submission is committed via a `useIssueCertificate` mutation in `lib/mutations/certificates.ts`. Previously issued certificates are listed below and each opens a dedicated route (`teacher/IjazaCertificate.tsx`) that renders the certificate as a printable document: ornamented border, the teacher's and student's names, the surah, the riwaya, the issue date, and a unique reference identifier. The page is print-stylesheet aware, so `Cmd/Ctrl + P` produces a clean, single-page A4 output.

Screenshot: Teacher Ijaza — issue a certificate and list past certificates

Screenshot: Ijaza Certificate detail — printable certificate view

Messages

Direct messaging between teacher and student. The left column lists the teacher's students; selecting one opens the conversation thread. Messages are stored in Supabase and the page subscribes to a real-time channel (`useMessagesRealtime`) so incoming replies appear without a manual refresh. Outgoing messages use a `useSendMessage` mutation that optimistically appends the message and rolls back on error.

Screenshot: Teacher Messages — real-time chat with assigned students

3.4.10 Demo and Stub UI Disclosure

Several controls were retained from the visual prototype to preserve UX parity even though their backend wiring is still in progress. They are kept visible so the design intent is preserved, but the user-facing copy and behaviour clearly mark them as inert until the corresponding feature ships:

Table 3.8: Currently inert (stub) UI elements

Element	**Status**
AI metric cards on Admin Overview	Render fixed values from a `DEMO_METRICS` constant until the Go server's metrics endpoint is consumed
Email column on Admin Users	Visual placeholder until the protected `auth.users` table is exposed via an RLS-safe view
Send Feedback button (Recitation Review)	Awaiting the feedback-mutation endpoint
Override (Tajweed annotation)	Local-only override; persistence pending the teacher-overrides table
Print / Export PDF on certificates	Falls back to the browser's print stylesheet for now
Grant / Revoke subscription tier	Awaiting the billing webhook integration

3.4.11 Testing

The dashboard ships a Vitest + React Testing Library setup under `tests/`. `jsdom` provides the browser-like environment, `@testing-library/user-event` drives user interactions, and `@testing-library/jest-dom` adds the standard DOM matchers. The suite is executed with `npm test` (one-shot) or `vitest -watch` during development and runs in CI alongside `npm run typecheck` and `npm run lint`.

3.5 Conclusion

This chapter presented the full technology stack powering the Tahkik platform and the realised interfaces of both the mobile application and the web dashboard. The Flutter app combines Riverpod, GoRouter, Dio, `web_socket_channel` and the custom KFGQPC Warsh font to deliver a complete learner journey from onboarding to subscription. The React dashboard layers TanStack Query, Recharts, CSS Modules and Sonner on top of Supabase to give administrators and teachers a unified, live, and accessible operations console. The Go gateway and the fine-tuned Whisper inference service form the AI backbone — a backbone whose internals, training procedure, and deployment topology are detailed in the two companion technical reports appended to this dissertation.

General Conclusion

The primary objective of this Final Year Project was to design, develop, and deploy a comprehensive digital platform dedicated to the learning and perfection of Quranic recitation and Tajweed rules. Through the development of **Tahkik**, we have demonstrated how modern technology — specifically Artificial Intelligence, cross-platform mobile development, and cloud-based backend services — can be leveraged to address the traditional challenges of accessibility and personalized feedback in Islamic education.

In Chapter 1, we presented the domain of Quranic recitation and Tajweed, analyzed existing digital solutions including Quran.com, Tarteel AI, and online tutoring platforms, and identified a critical gap: the absence of a platform combining instant AI feedback with structured human teacher review.

In Chapter 2, we modeled the system using UML to ensure a clear separation of concerns among the different actors (Learners, Teachers, Administrators), defined 10 functional and 5 non-functional requirements, and designed the system architecture with detailed use case, sequence, and class diagrams.

In Chapter 3, we successfully implemented the platform using Flutter for the mobile app, React for the web dashboard, Supabase for the backend, and Python with the Whisper model for AI-powered recitation analysis.

Key Contributions

1. A cross-platform mobile application providing an intuitive Quran reading and recitation recording experience.
2. Integration of the Whisper AI model for real-time pronunciation analysis and Tajweed error detection.
3. A dedicated web dashboard empowering teachers to review, correct, and manage student recitations.
4. A comprehensive administration panel with analytics, user management, and AI health monitoring.
5. A human-in-the-loop architecture combining AI efficiency with teacher expertise.

Despite these achievements, certain limitations remain:

- The AI model's accuracy varies across different recitation styles (Qira'at) and regional accents.
- The current implementation requires an internet connection for AI processing.
- The gamification features and social interaction elements are limited in this version.

Future Perspectives

- Fine-tuning the Whisper model specifically on diverse Qira'at (Warsh, Hafs, etc.) for improved accuracy.
- Adding offline AI inference capabilities using on-device models.
- Implementing gamification features (badges, leaderboards, learning paths) to increase engagement.
- Integrating live tutoring sessions with built-in video/audio calls between teachers and students.
- Developing community features for learners to share achievements and support each other.

Closing Statement

"Read! In the name of your Lord who created."

— Surah Al-Alaq, Verse 1

The Tahkik platform represents a meaningful step forward in digital Islamic education. By harmonizing the efficiency of AI with the irreplaceable expertise of human teachers, Tahkik aspires to make high-quality Tajweed instruction accessible to a global audience, preserving the oral tradition of the Quran for generations to come.

Bibliography

1. OpenAI, "Whisper: Robust Speech Recognition via Large-Scale Weak Supervision," 2022. Available: `https://openai.com/research/whisper`
2. Google, "Flutter — Build apps for any screen," 2024. Available: `https://flutter.dev`
3. Meta, "React — A JavaScript library for building user interfaces," 2024. Available: `https://react.dev`
4. Supabase, "Supabase — The open source Firebase alternative," 2024. Available: `https://supabase.com`
5. Riverpod, "Riverpod — A reactive caching and data-binding framework," 2024. Available: `https://riverpod.dev`
6. PyTorch, "PyTorch — An open source machine learning framework," 2024. Available: `https://pytorch.org`
7. Microsoft, "Visual Studio Code — Code Editing. Redefined," 2024. Available: `https://code.visualstudio.com`
8. Git, "Git — Distributed Version Control System," 2024. Available: `https://git-scm.com`
9. GitHub, "GitHub — Where the world builds software," 2024. Available: `https://github.com`
10. Vite, "Vite — Next Generation Frontend Tooling," 2024. Available: `https://vitejs.dev`
11. TanStack, "TanStack Query — Powerful asynchronous state management," 2024. Available: `https://tanstack.com/query`
12. A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
13. S. Al-Fedaghi, "Unified Modeling Language (UML) Sequence Diagrams: A Systematic Review," *International Journal of Software Engineering*, vol. 12, no. 3, pp. 45–62, 2021.

14. K. Schwaber and J. Sutherland, “The Scrum Guide,” Scrum.org, 2020.